# GUÍA DE LABORATORIO

# (formato docente)

## INFORMACIÓN BÁSICA

| | | | | | |
|---|---|---|---|---|---|
| **ASIGNATURA:** | CONSTRUCCION DE SOFTWARE | | | | |
| **TÍTULO DE LA PRÁCTICA:** | **Arquitectura como modelo: el modelo C4 (contexto, contenedores, componentes, dinámico y despliegue) con Structurizr DSL, C4-PlantUML, Mermaid e IA** | | | | |
| **NÚMERO DE PRÁCTICA:** | 06 | **AÑO LECTIVO:** | 2026B | **NRO. SEMESTRE:** | VI |
| **TIPO DE PRÁCTICA:** | **INDIVIDUAL** | | | | |
| | **GRUPAL** | **X** | **MÁXIMO DE ESTUDIANTES** | | 3 |
| **FECHA INICIO:** | 12/10/2026 | **FECHA FIN:** | 17/10/2026 | **DURACIÓN:** | 90 min presenciales + trabajo autónomo |

**RECURSOS A UTILIZAR:**

*Computador con acceso a Internet; repositorio de los Laboratorios 04 y 05 en GitHub; Java 17+ y Structurizr CLI (o Docker con Structurizr Lite); Visual Studio Code con las extensiones PlantUML y Markdown Preview Mermaid Support; PlantUML con la biblioteca C4-PlantUML; Mermaid Live Editor (mermaid.live); sitio oficial https://c4model.com; asistente de IA generativa (Claude, ChatGPT, Gemini o GitHub Copilot); aula virtual del curso.*

**DOCENTE(s):**

*Mg. Antonio Arroyo Paz*

## OBJETIVOS/TEMAS Y COMPETENCIAS

**OBJETIVOS:**

1. Explicar las abstracciones y los niveles del modelo C4 (contexto, contenedores, componentes y código) y su relación con los ADR y los diagramas UML.
2. Modelar un sistema real en los niveles de contexto, contenedores y componentes, junto con sus vistas dinámica y de despliegue, de forma coherente con los Laboratorios 04 y 05.
3. Aplicar "arquitectura como código" con un modelo único (Structurizr DSL) del que se derivan todas las vistas, y compararlo con C4-PlantUML y Mermaid.
4. Evaluar la calidad de los diagramas con la lista de revisión oficial del modelo C4 y con apoyo crítico de la IA, y comunicarlos a públicos técnicos y no técnicos.

**TEMAS:**

- Problemas de los diagramas de "cajas y flechas" y propósito del modelo C4 (Simon Brown).
- Abstracciones: persona, sistema de software, contenedor, componente y código.
- Diagramas principales (niveles 1 a 4) y complementarios (panorama del sistema, dinámico y despliegue).
- Notación, leyenda y lista de revisión oficial.
- Herramientas: Structurizr DSL (modelado), C4-PlantUML y Mermaid (diagramación), y uso responsable de la IA.

| | |
|---|---|
| **COMPETENCIAS** | **C.b:** Genera de forma responsable prototipos, experimentos y modelos, con el fin de analizar e interpretar información para la toma de decisiones fundamentadas y objetivas. |

| | |
|---|---|
| | **C.c:** Diseña responsablemente sistemas, componentes o procesos para satisfacer necesidades dentro de restricciones realistas. |
| | **C.d:** Trabaja éticamente, de manera efectiva, en equipos multidisciplinarios, adaptándose a diferentes entornos y nuevas situaciones. |
| | **C.g:** Comunica con claridad y de forma efectiva ideas, resultados y productos relacionados con su ejercicio profesional. |

## CONTENIDO DE LA GUÍA

# I. MARCO CONCEPTUAL

## 1.1 ¿Por qué el modelo C4?

Muchos diagramas de arquitectura son "cajas y flechas" ambiguos: no se sabe si una caja es un servidor, una aplicación o un módulo, ni qué significa cada flecha o color. El **modelo C4**, creado por Simon Brown, propone primero un conjunto pequeño de **abstracciones** y luego **diagramas jerárquicos**, como hacer zoom en un mapa: cada nivel muestra más detalle de una parte del anterior. Es **independiente de la notación** y **de la herramienta** (c4model.com).

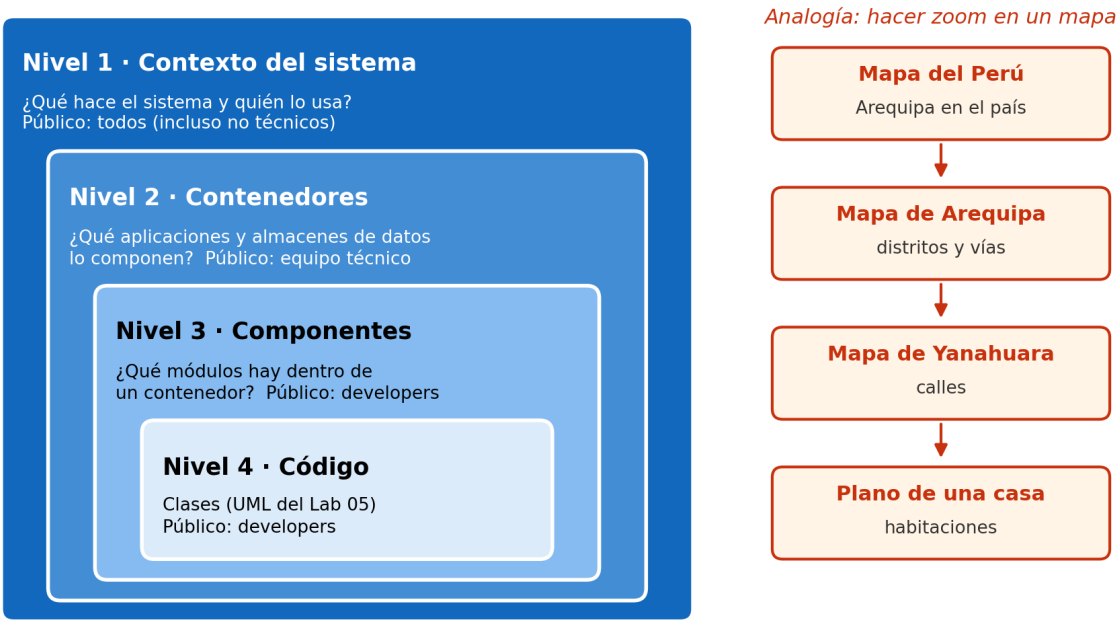



***Figura 1.*** *Los cuatro niveles del modelo C4 y la analogía del zoom en un mapa. Gráfico elaborado con Python (matplotlib).*

## 1.2 Abstracciones del modelo C4

**Tabla 1.** Abstracciones del modelo C4

| Abstracción | Definición | Ejemplo en AgroConecta |
|---|---|---|
| Persona | Usuario humano del sistema (actor, rol o perfil). | Productor, comprador, administrador de la cooperativa. |
| Sistema de software | Lo que entrega valor a sus usuarios; el nivel más alto de abstracción. Puede ser propio o externo. | AgroConecta (propio); proveedor de pagos y WhatsApp (externos). |
| Contenedor | Aplicación o almacén de datos que se ejecuta o despliega por separado y es necesario para que el sistema funcione. | Aplicación web progresiva, API, worker, PostgreSQL, Redis. |
| Componente | Agrupación de funcionalidad dentro de un contenedor, con una interfaz bien definida. | Módulos Catálogo, Pedidos, Pagos, Notificaciones y Reportes. |

| | | |
|---|---|---|
| Código | Clases, interfaces y funciones que implementan un componente. | Pedido, LineaPedido, PasarelaPago (diagrama de clases del Laboratorio 05). |

**Atención: "contenedor" en C4 NO significa contenedor Docker**

En C4, un contenedor es cualquier unidad que se ejecuta o despliega por separado: una aplicación web, una API, una app móvil, una base de datos, una cola o incluso un script programado. Puede ejecutarse o no dentro de Docker. Dónde se ejecuta cada contenedor se muestra en el diagrama de DESPLIEGUE, no en el de contenedores.

## 1.3 Diagramas principales y complementarios

**Tabla 2.** Diagramas del modelo C4 (adaptado de c4model.com)

| Diagrama | Alcance | Muestra | Público | ¿Cuándo? |
|---|---|---|---|---|
| 1. Contexto del sistema | Un sistema de software | El sistema, sus usuarios y los sistemas externos | Todos, incluso no técnicos | Siempre |
| 2. Contenedores | Un sistema de software | Los contenedores y la tecnología de cada uno | Equipo técnico | Siempre |
| 3. Componentes | Un contenedor | Los componentes internos y sus responsabilidades | Arquitectos y developers | Si aporta valor |
| 4. Código | Un componente | Clases e interfaces (UML) | Developers | Rara vez; mejor generarlo desde el código |
| Panorama del sistema | La organización | Todos los sistemas y personas de la empresa | Todos | Organizaciones con varios sistemas |
| Dinámico | Un escenario | Cómo colaboran los elementos en un caso de uso, con pasos numerados | Equipo técnico | Flujos críticos |
| Despliegue | Un entorno | Dónde se ejecuta cada contenedor (servidores, nube, dispositivos) | Equipo técnico y operaciones | Siempre que haya producción |

## 1.4 Notación y lista de revisión

C4 no impone formas ni colores, pero exige que el diagrama **se explique por sí mismo**. La lista de revisión oficial de c4model.com se resume así:

**Tabla 3.** Lista de revisión de diagramas de arquitectura (resumen de c4model.com/diagrams/checklist)

| Grupo | Preguntas de revisión |
|---|---|
| General | ¿Tiene título? ¿Se entiende de qué tipo de diagrama se trata y cuál es su alcance? ¿Tiene leyenda? |
| Elementos | ¿Cada elemento tiene nombre, tipo (persona, sistema, contenedor o componente), descripción de lo que hace y, si corresponde, tecnología? ¿Se entienden los acrónimos, colores, formas, íconos, bordes y tamaños? |
| Relaciones | ¿Cada flecha tiene una etiqueta que describe su intención? ¿La etiqueta coincide con la dirección de la flecha? ¿Se indica la tecnología o el protocolo (HTTPS, SQL, MQTT)? ¿Se entienden los estilos de línea y las puntas de flecha? |

## 1.5 Herramientas: diagramar o modelar

Hay dos enfoques. En una herramienta de **diagramación** (C4-PlantUML, Mermaid, draw.io) cada diagrama es un archivo independiente; si un contenedor cambia de nombre, hay que corregirlo en todos los diagramas. En una herramienta de **modelado** (Structurizr DSL, LikeC4, IcePanel) se describe **un solo modelo** y las vistas se derivan de él, por lo que son siempre coherentes.

**Tabla 4.** Herramientas para el modelo C4

| Herramienta | Enfoque | Fortaleza | Uso en esta guía |
|---|---|---|---|
| Structurizr DSL | Modelado | Un modelo, muchas vistas; exporta a PlantUML, Mermaid y otros formatos; integra ADR y documentación. | Fuente principal (workspace.dsl) |
| C4-PlantUML | Diagramación | Macros C4 (Person, System, Container, Rel) sobre PlantUML; incluida en la biblioteca estándar. | Comparación y exportación |
| Mermaid C4 | Diagramación | Sintaxis similar a C4-PlantUML; GitHub la renderiza en el README (soporte experimental). | Diagrama embebido en el README |
| LikeC4 / IcePanel | Modelado | Editores modernos con navegación interactiva entre niveles. | Exploración opcional |

**C4-PlantUML — ejemplo de diagrama de contexto escrito a mano**

```
@startuml contexto-manual
!include <C4/C4_Context>
skinparam dpi 180
LAYOUT_LEFT_RIGHT()
title Contexto de AgroConecta (C4-PlantUML escrito a mano)

Person(productor, "Productor", "Pequeño productor de Majes")
Person(comprador, "Comprador", "Restaurante, bodega o mercado")
System(agro, "AgroConecta", "Publicación de producción, pedidos y pagos")
System_Ext(pagos, "Proveedor de pagos", "Yape / Plin")
System_Ext(wa, "WhatsApp Business API", "Mensajería")

Rel(productor, agro, "Publica su producción")
Rel(comprador, agro, "Hace pedidos y paga")
Rel(agro, pagos, "Solicita cobros", "HTTPS/JSON")
Rel(agro, wa, "Envía notificaciones", "HTTPS/JSON")
SHOW_LEGEND()
@enduml
```

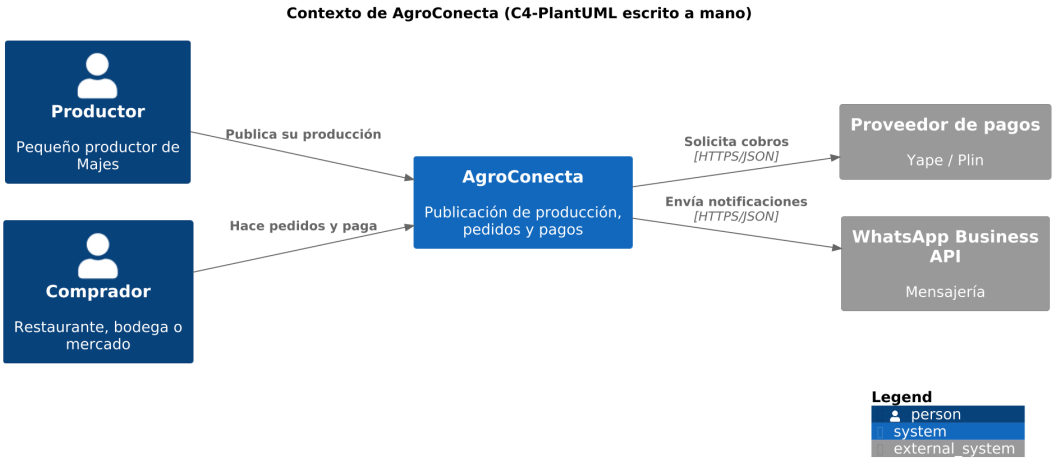



***Figura 2.*** *Diagrama de contexto escrito con macros de C4-PlantUML.*

**Mermaid — ejemplo de diagrama de contenedores (C4Container)**

```
C4Container
    title Contenedores de AgroConecta (Mermaid C4)
    Person(comprador, "Comprador", "Restaurante o bodega")
    System_Boundary(agro, "AgroConecta") {
        Container(pwa, "PWA", "React", "Interfaz web y móvil")
        Container(api, "API", "Django REST", "Monolito modular")
        ContainerDb(db, "Base de datos", "PostgreSQL", "Pedidos y pagos")
    }
    System_Ext(pagos, "Proveedor de pagos", "Yape / Plin")
    Rel(comprador, pwa, "Usa", "HTTPS")
    Rel(pwa, api, "Llama a", "JSON/HTTPS")
    Rel(api, db, "Lee y escribe", "SQL")
    Rel(api, pagos, "Solicita cobros", "HTTPS")
    UpdateLayoutConfig($c4ShapeInRow="3", $c4BoundaryInRow="1")
```

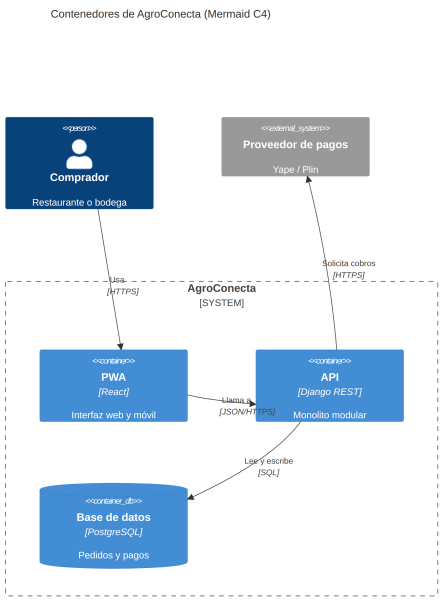



***Figura 3.*** *El mismo enfoque con Mermaid C4, que GitHub muestra directamente en un archivo Markdown.*

## 1.6 El modelo C4 en la secuencia de laboratorios

**Tabla 5.** Cómo se conectan los Laboratorios 04, 05 y 06

| Laboratorio | Pregunta que responde | Artefactos | Relación con C4 |
|---|---|---|---|
| 04 Arquitectura | ¿Por qué el sistema es así? | Drivers, matriz de decisión, ADR, despliegue.py | Los ADR justifican los contenedores; despliegue.py anticipa la vista de despliegue. |
| 05 UML | ¿Cómo funciona por dentro? | Clases, secuencia, estados, paquetes | Clases = nivel 4 (código); paquetes ≈ nivel 3; secuencia ≈ vista dinámica. |
| 06 C4 | ¿Qué hay y cómo se conecta, a distintos niveles de zoom? | workspace.dsl y 5 vistas | Integra todo en un solo modelo navegable. |

## 1.7 Flujo de trabajo con IA

La IA es muy útil para redactar un primer borrador del modelo en Structurizr DSL a partir de los ADR y diagramas existentes, para auditar los diagramas con la lista de revisión y para explicar la arquitectura a públicos no técnicos. Sus errores típicos en C4 son: **confundir contenedor con contenedor Docker**, poner como contenedores internos a sistemas externos, **inventar tecnologías** (Kafka, Kubernetes) que no están en el ADR y dejar flechas sin etiqueta.

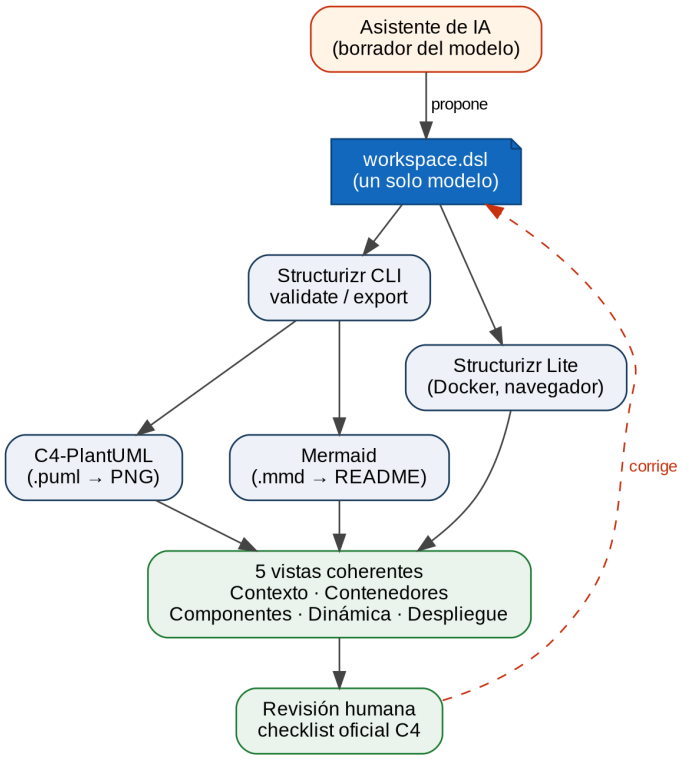



***Figura 4.*** *Flujo "arquitectura como código" con un modelo único y revisión humana. Diagrama generado con Graphviz.*

## II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

**Caso: AgroConecta Majes (continuación de los Laboratorios 04 y 05).** Arquitectura elegida: **monolito modular** en Django (ADR-001), con aplicación web progresiva, worker Celery, PostgreSQL y Redis, desplegados en un VPS con Docker Compose. El MVP debe salir en **1 mes**.

### Paso 1. Inventario de elementos a partir de lo ya construido

**Tabla 6.** inventario.md: elementos del modelo y su origen

| Elemento | Tipo C4 | Origen |
|---|---|---|
| Productor, comprador, administrador | Persona | drivers.md (Lab 04) |
| AgroConecta | Sistema de software | Caso |
| Proveedor de pagos, WhatsApp Business API | Sistema externo | drivers.md, ADR-001 |
| Aplicación web progresiva, API, worker, base de datos, cola y caché | Contenedor | ADR-001 y despliegue.py (Lab 04) |
| Módulos Catálogo, Pedidos, Pagos, Notificaciones, Reportes | Componente (de la API) | paquetes.puml (Lab 05) |
| Flujo "confirmar y pagar" | Vista dinámica | secuencia-confirmar-pagar.puml (Lab 05) |

### Paso 2. Generar el borrador del modelo con IA

**PROMPT IA 1 — Borrador del modelo en Structurizr DSL**

Actúa como arquitecto de software experto en el modelo C4 y en Structurizr DSL.
Contexto: [pegar inventario.md, ADR-001 y despliegue.py].

Tarea: escribe un workspace de Structurizr DSL con personas, el sistema, sus contenedores (con tecnología), los componentes del contenedor API, los sistemas externos (con la etiqueta "Externo") y relaciones con descripción y protocolo. Incluye las vistas systemContext, container, component, dynamic (confirmar y pagar) y deployment (entorno "Producción").
Restricciones: usa SOLO las tecnologías del ADR-001; no confundas contenedor C4 con contenedor Docker; usa !identifiers hierarchical. Formato: solo el código DSL.

**Tabla 7.** Bitácora de IA: verificación del borrador

| Propuesta de la IA | Decisión | Justificación |
|---|---|---|
| Un contenedor llamado "Docker Compose" dentro de AgroConecta | Corregida | Docker Compose es infraestructura: va como nodo en la vista de despliegue, no como contenedor. |
| "WhatsApp" como contenedor interno | Corregida | Es un sistema externo que no construimos ni desplegamos. |
| Un broker Kafka para las notificaciones | Rechazada | No figura en el ADR-001; el diseño usa Redis + Celery (y Kafka excede el plazo de 1 mes). |
| Relaciones sin protocolo | Corregida | Se agregaron HTTPS/JSON, SQL/TCP y Redis protocol, según la lista de revisión. |

## Paso 3. Modelo final en Structurizr DSL

**Structurizr DSL — docs/architecture/c4/workspace.dsl**

```
workspace "AgroConecta Majes" "Modelo C4 de la plataforma de la Cooperativa Agraria de Majes" {

    !identifiers hierarchical

    model {
        productor = person "Productor" "Pequeño productor de leche y hortalizas de Majes."
        comprador = person "Comprador" "Restaurante, bodega o mercado de Arequipa."
        admin = person "Administrador de la cooperativa" "Supervisa ventas y productores."

        agro = softwareSystem "AgroConecta" "Permite publicar la producción diaria, hacer pedidos y
pagarlos." {
            pwa = container "Aplicación web progresiva" "Interfaz para productores y compradores;
funciona con conectividad intermitente." "React + Service Worker" "Web Browser"
            api = container "API AgroConecta" "Monolito modular que implementa los casos de uso
mediante una API REST." "Python / Django REST Framework" {
                catalogo = component "Módulo Catálogo" "Publica ofertas y gestiona el stock." "Django
app"
                pedidos = component "Módulo Pedidos" "Carrito, confirmación y ciclo de vida del
pedido." "Django app"
                pagos = component "Módulo Pagos" "Cobra mediante el puerto PasarelaPago." "Django
app"
                notificaciones = component "Módulo Notificaciones" "Envía avisos a productores y
compradores." "Django app"
                reportes = component "Módulo Reportes" "Reportes de ventas para la cooperativa."
"Django app"
            }
            worker = container "Worker de tareas" "Ejecuta notificaciones y reintentos de forma
asíncrona." "Celery"
            db = container "Base de datos" "Usuarios, ofertas, pedidos y pagos (un esquema por
módulo)." "PostgreSQL 16" "Database"
            cola = container "Cola y caché" "Cola de tareas y caché del catálogo." "Redis" "Queue"
        }
```

```
    pasarela = softwareSystem "Proveedor de pagos" "Procesa cobros con Yape y Plin." "Externo"
    whatsapp = softwareSystem "WhatsApp Business API" "Envía mensajes a los usuarios." "Externo"

    productor -> agro "Publica su producción y atiende pedidos"
    comprador -> agro "Busca ofertas, hace pedidos y paga"
    admin -> agro "Consulta reportes de ventas"
    agro -> pasarela "Solicita cobros" "HTTPS/JSON"
    agro -> whatsapp "Envía notificaciones" "HTTPS/JSON"

    productor -> agro.pwa "Usa" "HTTPS"
    comprador -> agro.pwa "Usa" "HTTPS"
    admin -> agro.pwa "Usa" "HTTPS"
    agro.pwa -> agro.api "Llama a" "HTTPS/JSON (REST)"
    agro.api -> agro.db "Lee y escribe" "SQL/TCP"
    agro.api -> agro.cola "Encola tareas y lee caché" "Redis protocol"
    agro.worker -> agro.cola "Consume tareas" "Redis protocol"
    agro.worker -> whatsapp "Envía mensajes" "HTTPS/JSON"
    agro.api -> pasarela "Solicita cobros" "HTTPS/JSON"

    agro.pwa -> agro.api.catalogo "Consulta ofertas" "HTTPS/JSON"
    agro.pwa -> agro.api.pedidos "Gestiona pedidos" "HTTPS/JSON"
    agro.pwa -> agro.api.reportes "Solicita reportes" "HTTPS/JSON"
    agro.api.pedidos -> agro.api.catalogo "Reserva stock"
    agro.api.pedidos -> agro.api.pagos "Solicita el cobro"
    agro.api.pedidos -> agro.api.notificaciones "Pide notificar"
    agro.api.reportes -> agro.api.pedidos "Lee pedidos (solo lectura)"
    agro.api.pagos -> pasarela "Cobra" "HTTPS/JSON"
    agro.api.notificaciones -> agro.cola "Encola mensajes" "Redis protocol"
    agro.api.catalogo -> agro.db "Lee y escribe" "SQL"
    agro.api.pedidos -> agro.db "Lee y escribe" "SQL"

    produccion = deploymentEnvironment "Producción" {
        deploymentNode "Celular o PC del usuario" "" "Android / navegador web" {
            containerInstance agro.pwa
        }
        deploymentNode "VPS en la nube" "" "Ubuntu 24.04, 2 vCPU, 4 GB RAM" {
            deploymentNode "Docker Compose" "" "Docker 27" {
                containerInstance agro.api
                containerInstance agro.worker
                containerInstance agro.db
                containerInstance agro.cola
            }
        }
    }
}

views {
    systemContext agro "Contexto" "Nivel 1: AgroConecta y su entorno." {
        include *
        autolayout lr
    }
    container agro "Contenedores" "Nivel 2: contenedores de AgroConecta." {
        include *
        autolayout lr
    }
    component agro.api "Componentes" "Nivel 3: módulos de la API." {
        include *
        autolayout lr
    }
    dynamic agro.api "ConfirmarYPagar" "Dinámico: confirmar y pagar un pedido." {
        agro.pwa -> agro.api.pedidos "POST /pedidos/{id}/pago"
        agro.api.pedidos -> agro.api.catalogo "Reserva stock"
```

```
            agro.api.pedidos -> agro.api.pagos "Solicita el cobro"
            agro.api.pagos -> pasarela "Cobra"
            agro.api.pedidos -> agro.api.notificaciones "Pide notificar"
            agro.api.notificaciones -> agro.cola "Encola mensajes"
            autolayout lr
        }
        deployment agro produccion "Despliegue" "Despliegue en producción (MVP)." {
            include *
            autolayout lr
        }
        styles {
            element "Person" {
                shape person
                background #08427B
                color #ffffff
            }
            element "Software System" {
                background #1168BD
                color #ffffff
            }
            element "Externo" {
                background #999999
            }
            element "Container" {
                background #438DD5
                color #ffffff
            }
            element "Component" {
                background #85BBF0
                color #000000
            }
            element "Database" {
                shape cylinder
            }
            element "Queue" {
                shape pipe
            }
            element "Web Browser" {
                shape WebBrowser
            }
        }
    }
}
```

## Paso 4. Validar, exportar y visualizar

**Bash — Structurizr CLI y Structurizr Lite**

```
# Validar el modelo (detecta identificadores inexistentes y errores de sintaxis)
./structurizr.sh validate -w workspace.dsl

# Exportar todas las vistas a C4-PlantUML y a Mermaid
./structurizr.sh export -w workspace.dsl -f plantuml/c4plantuml -o export
./structurizr.sh export -w workspace.dsl -f mermaid -o export

# Generar las imágenes PNG
java -jar plantuml.jar -tpng -o ../img export/*.puml

# Opción interactiva: Structurizr Lite en http://localhost:8080
docker run -it --rm -p 8080:8080 -v "$PWD":/usr/local/structurizr structurizr/lite
```

Las cinco figuras siguientes se generaron **desde el mismo archivo** `workspace.dsl`: si se cambia el nombre o la tecnología de un contenedor, todas las vistas se actualizan al volver a exportar.

## Paso 5. Nivel 1: diagrama de contexto del sistema

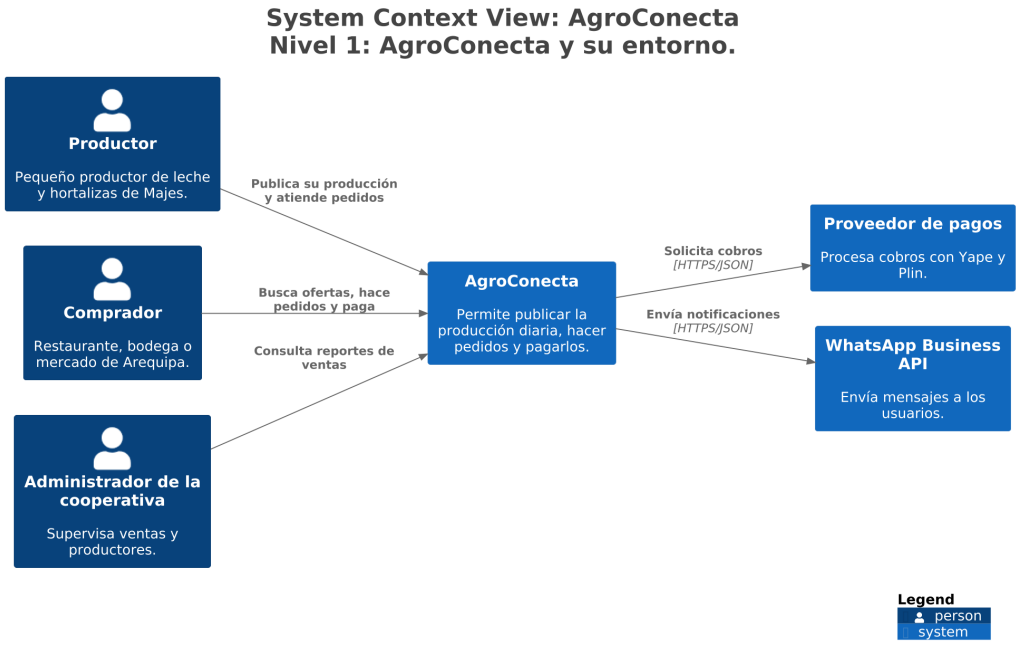



***Figura 5.*** *Vista de contexto exportada de Structurizr DSL a C4-PlantUML.*

**Lectura:** una persona no técnica entiende qué hace AgroConecta, quién lo usa y de qué sistemas externos depende. No aparecen tecnologías internas.

## Paso 6. Nivel 2: diagrama de contenedores

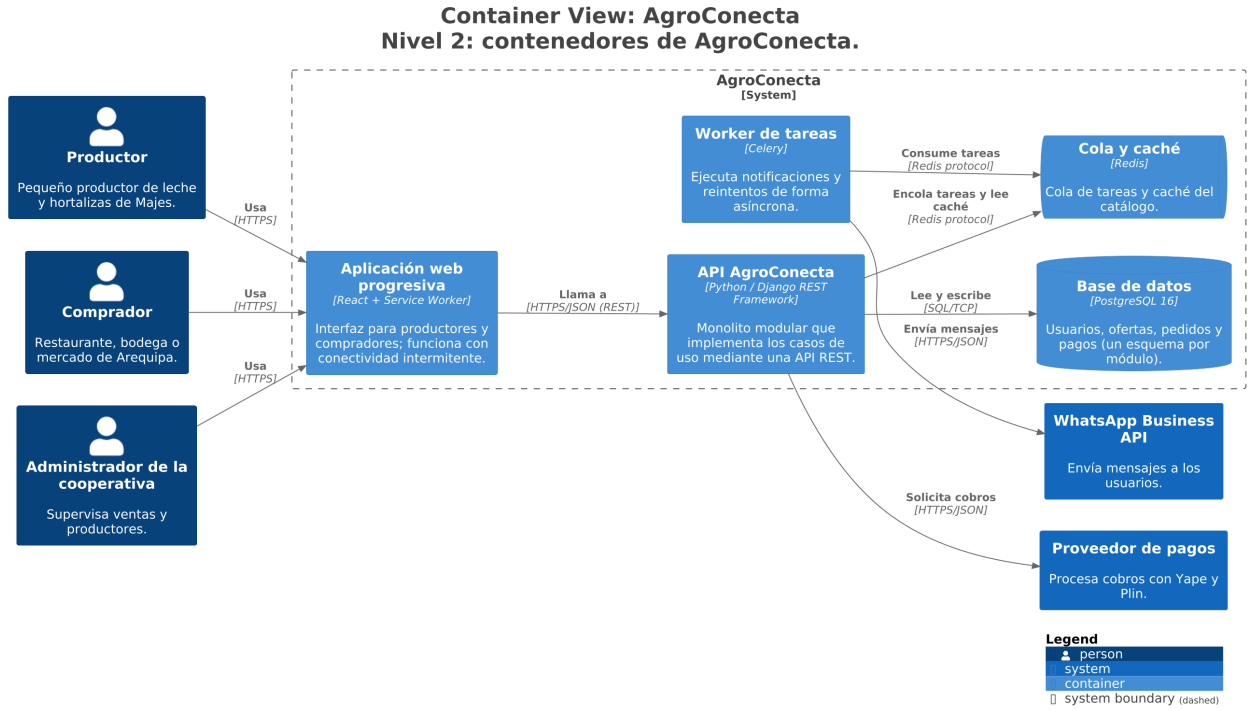



***Figura 6.*** *Vista de contenedores: cada caja indica su tecnología y cada flecha, su protocolo.*

**Lectura:** se ven las decisiones del ADR-001: un solo contenedor de aplicación (monolito modular), procesamiento asíncrono con worker y Redis, y una base de datos PostgreSQL.

## Paso 7. Nivel 3: diagrama de componentes de la API

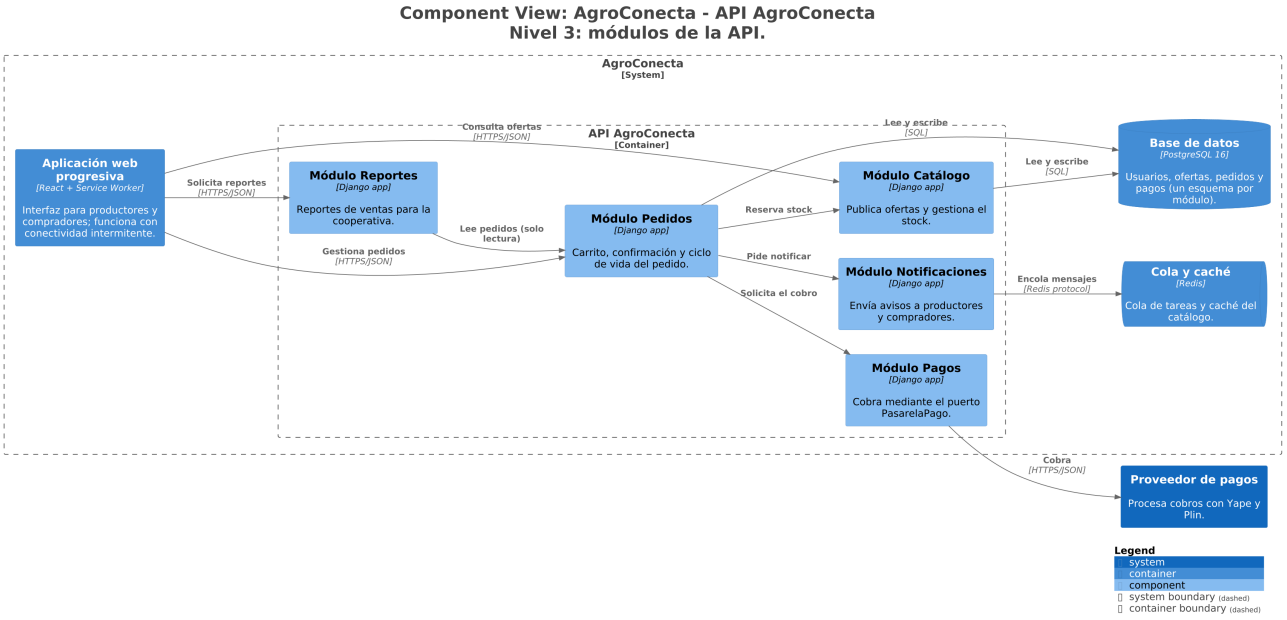



***Figura 7.*** *Vista de componentes del contenedor API: coincide con el diagrama de paquetes del Laboratorio 05.*

**Verificación cruzada:** las dependencias entre componentes (Pedidos usa Catálogo, Pagos y Notificaciones; Reportes solo lee Pedidos) son las mismas del diagrama de paquetes del Laboratorio 05. El nivel 4 (código) **no se dibuja de nuevo**: se enlaza el diagrama de clases `docs/design/clases.puml`.

## Paso 8. Vista dinámica: confirmar y pagar un pedido

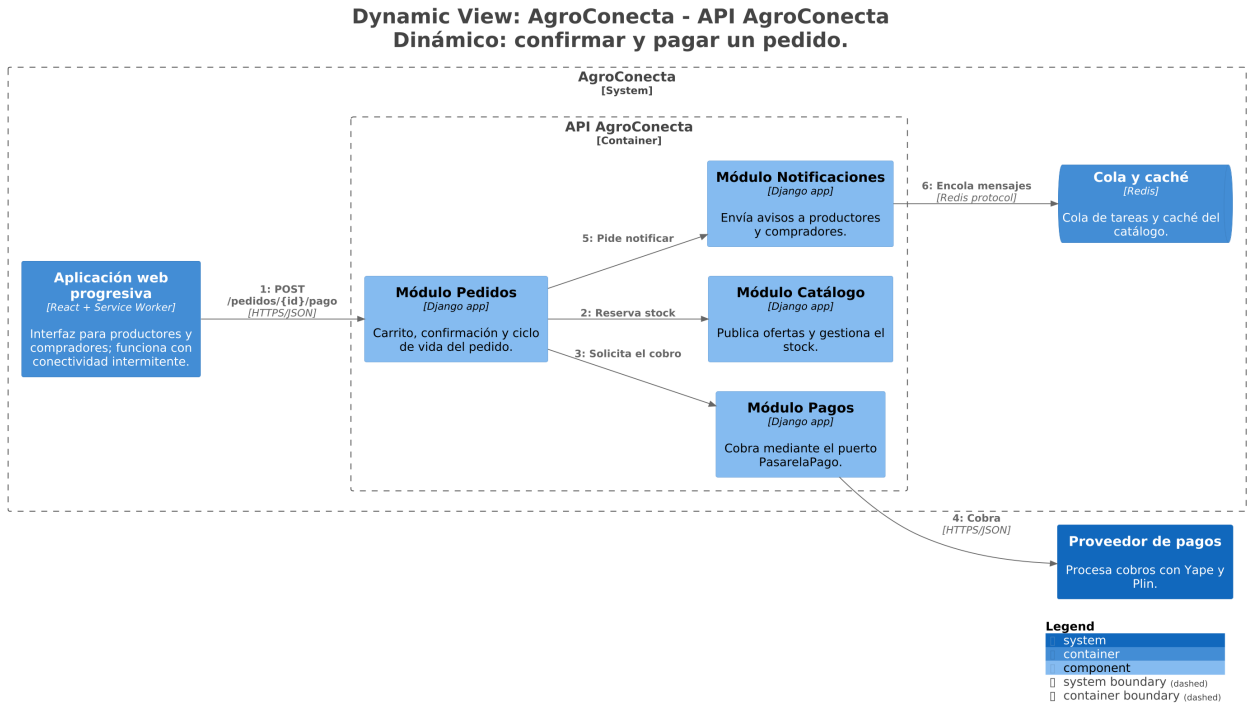



***Figura 8.*** *Vista dinámica con pasos numerados, equivalente de alto nivel del diagrama de secuencia del Laboratorio 05.*

## Paso 9. Vista de despliegue

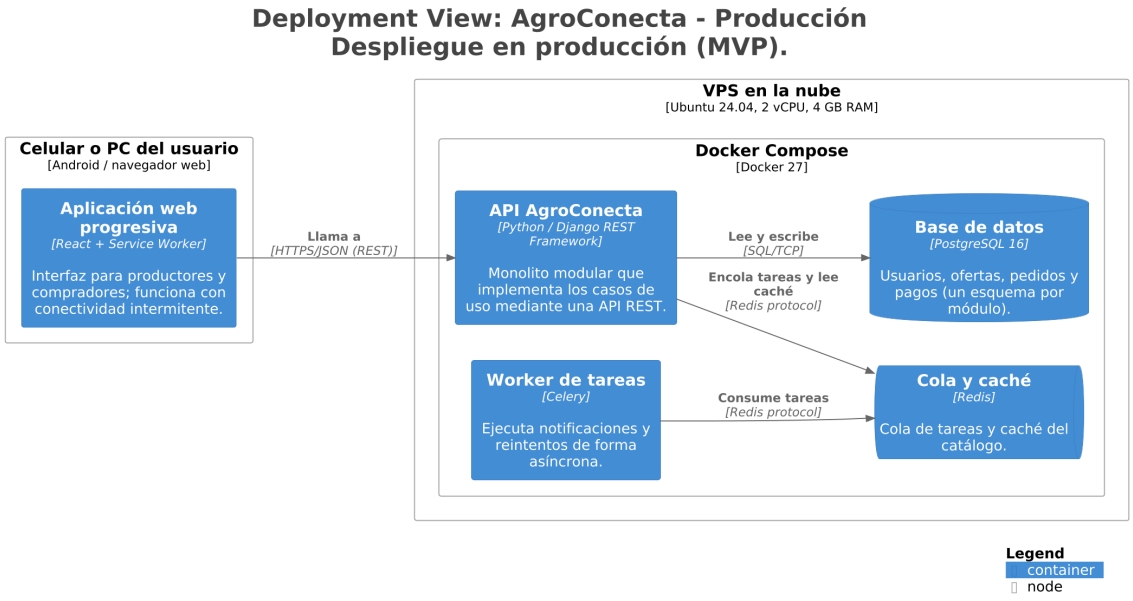



***Figura 9.*** *Vista de despliegue del entorno de producción: el VPS, Docker Compose y el dispositivo del usuario.*

## Paso 10. Revisión con la lista oficial y con IA

**PROMPT IA 2 — Auditor C4**

Actúa como revisor de diagramas de arquitectura. Evalúa estas vistas exportadas de un modelo C4
[pegar los .puml o adjuntar las imágenes] con la lista de revisión de c4model.com: generales, elementos y relaciones.
Responde en una tabla (vista, pregunta de la lista, cumple sí/no, evidencia, corrección sugerida).
No inventes problemas: si algo cumple, indícalo.

**Tabla 8.** Extracto de revision-c4.md

| Vista | Hallazgo de la IA | Verificación del equipo |
|---|---|---|
| Contenedores | La flecha "Encola tareas y lee caché" mezcla dos intenciones. | Correcto: se aceptan ambas porque comparten protocolo; se documenta en la descripción. |
| Contexto | Faltan las tecnologías de las relaciones con las personas. | Falso positivo: en el nivel 1 no se exigen tecnologías para las personas. |
| Despliegue | No se indica la versión de PostgreSQL. | Correcto: la tecnología del contenedor dice "PostgreSQL 16"; se verificó que el export la muestra. |
| Todas | Hay leyenda, título y descripción en cada vista. | Correcto (cumple). |

## Paso 11. Comunicar la arquitectura a un público no técnico

**PROMPT IA 3 — Explicación para la cooperativa**

Explica el siguiente diagrama de contexto al presidente de la Cooperativa Agraria de Majes, que no es
técnico, en máximo 120 palabras: qué hace el sistema, quién lo usa y de qué servicios externos depende.
Evita la jerga técnica. [pegar la vista de contexto]

**explicacion.md (versión revisada por el equipo)**

```
AgroConecta es una aplicación que pueden usar desde el celular los productores, los
compradores y la administración de la cooperativa. Los productores publican cada día cuánta
leche u hortalizas tienen y a qué precio; los compradores eligen, piden y pagan con Yape o
Plin; y la cooperativa ve cuánto se vende. Para cobrar, AgroConecta se conecta con un
proveedor de pagos, y para avisar de cada pedido envía mensajes de WhatsApp. Si alguno de
esos servicios falla, los pedidos no se pierden: se guardan y se reintentan.
```

## Paso 12. Estructura del repositorio y contenido de cada archivo

Se trabaja en el **mismo repositorio** de los Laboratorios 04 y 05, en la carpeta `docs/architecture/c4/`. Los códigos **E1 a E8** son los **entregables** de los ejercicios propuestos (E1 = Ejercicio 1, etc.).

**Estructura del repositorio cs-2026b-lab04-grupoXX (continuación)**

```
cs-2026b-lab04-grupoXX/
├── README.md                       # E8: sección "Arquitectura C4 (Lab 06)"
└── docs/
    ├── architecture/
    │   ├── drivers.md, adr/ ...    # Laboratorio 04
    │   └── c4/                     # Laboratorio 06
```

```
    │       ├── inventario.md        # E1: elementos y su origen
    │       ├── workspace.dsl       # E1–E5: modelo único con todas las vistas
    │       ├── export/             # E6: .puml y .mmd generados por la CLI
    │       ├── img/                # E6: PNG de las vistas
    │       ├── contexto-manual.puml # E6: versión escrita a mano con C4-PlantUML
    │       ├── comparacion.md      # E6: modelado frente a diagramación
    │       ├── revision-c4.md      # E7: lista de revisión + hallazgos de la IA
    │       ├── explicacion.md      # E7: explicación para público no técnico
    │       └── bitacora-ia.md      # E7: interacciones con la IA
    └── design/                     # Laboratorio 05 (clases = nivel 4)
```

**Tabla 9.** Qué va dentro de cada archivo

| Código | Archivo | Contenido mínimo obligatorio | Modelo |
|---|---|---|---|
| E1 | inventario.md, workspace.dsl | Tabla de elementos con tipo C4 y origen; en el DSL: 2 o más personas, el sistema, 2 o más sistemas externos y la vista systemContext. | Pasos 1, 2 y 5 |
| E2 | workspace.dsl | 3 o más contenedores con tecnología; relaciones con descripción y protocolo; vista container. | Paso 6 |
| E3 | workspace.dsl | 4 o más componentes del contenedor principal, coherentes con paquetes.puml del Lab 05; vista component. | Paso 7 |
| E4 | workspace.dsl | Vista dynamic del flujo crítico (el mismo de la secuencia del Lab 05) con 4 o más pasos. | Paso 8 |
| E5 | workspace.dsl | deploymentEnvironment "Producción" con nodos anidados e instancias de todos los contenedores; vista deployment. | Paso 9 |
| E6 | export/, img/, contexto-manual.puml, comparacion.md | Exportación a C4-PlantUML y Mermaid, imágenes de las 5 vistas y comparación entre el diagrama escrito a mano y el exportado. | Paso 4 |
| E7 | revision-c4.md, explicacion.md, bitacora-ia.md | Lista de revisión aplicada a las 5 vistas; hallazgos de la IA con al menos 1 falso positivo; explicación de 120 palabras o menos; 5 o más interacciones con IA. | Pasos 10 y 11 |
| E8 | README.md + tag | Sección "Arquitectura C4 (Lab 06)" con el contexto en Mermaid y enlaces a las vistas; etiqueta v0.6-c4. | Sección III |

## III. EJERCICIOS/PROBLEMAS PROPUESTOS

**Organización:** se mantienen los **mismos grupos, casos y repositorio** de los Laboratorios 04 y 05. El modelo C4 debe ser coherente con su ADR-001 y con su diseño UML: si encuentran una contradicción, corrijan el artefacto equivocado y regístrenlo en la bitácora.

**Condiciones comunes**

- Plazo: el MVP debe estar en producción en 1 mes; modele lo que realmente se construirá en ese plazo.
- Fuente única: todas las vistas salen de workspace.dsl; no se dibujan diagramas sueltos (salvo contexto-manual.puml en E6).
- Regla de oro: la IA propone, el equipo decide y verifica; todo se registra en bitacora-ia.md.

## Pistas de modelado para cada caso

**Tabla 10.** Pistas de modelado C4 por caso (validar con su propio ADR-001)

| N.º | Caso | Personas | Sistemas externos probables | Contenedores probables (según su ADR) |
|---|---|---|---|---|
| 1 | CitaSalud Arequipa | Paciente, personal de admisión, médico | Mensajería (WhatsApp/SMS), servicio de consulta de DNI | PWA, API, base de datos, worker de recordatorios |
| 2 | PicanteríaYa | Cliente, picantería, repartidor | Pasarela de pagos, servicio de mapas, WhatsApp | App cliente, app repartidor, API, servicio de seguimiento en tiempo real, base de datos |
| 3 | AireMisti IoT | Ciudadano, analista municipal | Canal de alertas (SMS/Telegram), servicio de mapas | Broker MQTT, servicio de ingesta, base de datos de series de tiempo, tablero web, servicio de alertas |
| 4 | BiblioUNSA | Estudiante, bibliotecario | Sistema académico UNSA, autenticación institucional, correo | Web, API, base de datos, tarea programada de multas |
| 5 | VotoEPIS | Estudiante, comité electoral, auditor | Autenticación institucional, padrón del sistema académico | Web de votación, API, base de datos de votos separada del padrón, registro de auditoría |
| 6 | RutaSIT Arequipa | Pasajero, operador | GPS de los buses, servicio de mapas | Servicio de ingesta GPS, motor de tiempos de llegada, API pública, app de pasajeros, base de datos |
| 7 | San Camilo en Línea | Cliente, comerciante, repartidor | Pasarela Yape/Plin, WhatsApp | PWA, API, base de datos, worker de notificaciones |
| 8 | SismoReporta AQP | Ciudadano, brigadista, coordinador | Servicio de mapas, almacenamiento de fotos, SMS | App sin conexión, API de recepción, cola, base de datos geoespacial, tablero de coordinación |
| 9 | ChacraSmart Majes | Agricultor, técnico | Servicio meteorológico, SMS | Gateway IoT de campo, broker MQTT, servicio de control de riego, app móvil, base de datos |
| 10 | EcoRecicla AQP | Vecino, reciclador, municipalidad | Servicio de mapas y rutas, WhatsApp | App vecino, app reciclador, API, base de datos, tarea de optimización de rutas |

## Parte A — Sesión de laboratorio (90 minutos)

| Tiempo | Actividad | Producto / checkpoint |
|---|---|---|
| 0–10 min | Presentación del tema; instalación de Structurizr CLI o Docker | Comando validate ejecutado sobre el ejemplo |
| 10–25 min | Demostración del docente: Pasos 2, 3, 4 y 10 | Ejemplo exportado y renderizado en cada equipo |
| 25–45 min | E1. Inventario y nivel 1 (contexto) | inventario.md y vista systemContext con commit |
| 45–65 min | E2. Nivel 2 (contenedores) | Vista container validada |
| 65–80 min | E3. Nivel 3 (componentes del contenedor principal) | Vista component validada |
| 80–90 min | Checkpoint | validate sin errores y 3 vistas exportadas; el docente revisa con la lista de cotejo |

### E1. Inventario y diagrama de contexto (20 min)

1. Cree `docs/architecture/c4/inventario.md` con la tabla del Paso 1: cada persona, sistema, contenedor y componente, con su tipo C4 y el archivo de los Laboratorios 04 o 05 de donde proviene.
2. Adapte el **Prompt IA 1** con su inventario y su ADR-001 para obtener un borrador de `workspace.dsl`. Registre el prompt en la bitácora.
3. Revise el borrador con el Paso 2 como guía. Corrija en especial: contenedores que en realidad son infraestructura, sistemas externos modelados como internos y tecnologías que no están en su ADR.
4. Deje en el modelo: **2 o más personas**, el sistema, **2 o más sistemas externos** (con la etiqueta "Externo") y relaciones con descripción. Defina la vista `systemContext` con título y descripción.
5. Ejecute `structurizr.sh validate` hasta que no haya errores. Haga commit con el mensaje `E1: contexto C4`.

☐ **Lista de verificación:** el diagrama de contexto no muestra tecnologías internas; cada flecha tiene un verbo que describe la intención ("Solicita cobros", no "usa").

### E2. Diagrama de contenedores (20 min)

1. Agregue dentro del sistema **3 o más contenedores** (aplicaciones y almacenes de datos), cada uno con descripción y **tecnología** concreta.
2. Agregue las relaciones entre contenedores y con los sistemas externos, indicando el **protocolo** (HTTPS/JSON, SQL/TCP, MQTT, etc.).
3. Use etiquetas y estilos (`Database`, `Queue`, `Web Browser`) para que la forma de cada elemento comunique su tipo.
4. Defina la vista `container`, valide y exporte. Haga commit con el mensaje `E2: contenedores C4`.

☐ **Lista de verificación:** los contenedores coinciden con los del ADR-001; ningún contenedor se llama "Docker", "servidor" o "nube".

### E3. Diagrama de componentes (15 min)

1. Elija el contenedor principal (normalmente la API) y agregue **4 o más componentes**, que deben coincidir con los paquetes de `docs/design/paquetes.puml` del Laboratorio 05.
2. Modele las dependencias entre componentes y hacia contenedores o sistemas externos. Verifique que **no haya ciclos**, igual que en la regla C4 del Laboratorio 05.
3. Defina la vista `component`, valide y haga commit con el mensaje `E3: componentes C4`.

☐ **Lista de verificación:** si un componente no existe en paquetes.puml, actualice el diagrama UML o el modelo C4, y anote la corrección en la bitácora.

## Parte B — Trabajo autónomo (13/10/2026 al 17/10/2026)

### E4. Vista dinámica del flujo crítico

1. Tome el mismo flujo del diagrama de secuencia del Laboratorio 05.
2. Escriba una vista `dynamic` con **4 o más pasos** entre contenedores o componentes; cada paso debe corresponder a una relación ya existente en el modelo.
3. En `comparacion.md`, explique en 3 a 5 líneas qué detalle muestra la secuencia UML que la vista dinámica omite, y para qué público sirve cada una.

### E5. Vista de despliegue

1. Defina `deploymentEnvironment "Producción"` con nodos anidados: dispositivo del usuario, servidor o nube, Docker u otro entorno de ejecución.
2. Coloque una instancia (`containerInstance`) de **cada** contenedor del nivel 2 en algún nodo.
3. Indique sistema operativo, tamaño o servicio de nube de cada nodo, y compruebe que sea coherente con la restricción de presupuesto de su `drivers.md`.

### E6. Exportación y comparación de herramientas

1. Exporte todas las vistas a C4-PlantUML y a Mermaid en la carpeta `export/`, y genere los PNG en `img/`.
2. Escriba a mano `contexto-manual.puml` con las macros de C4-PlantUML (como en la sección 1.5) para el mismo contexto.
3. En `comparacion.md`, compare ambos enfoques (escritura a mano frente a modelo exportado) en: tiempo invertido, errores encontrados y qué pasaría si mañana cambia el nombre de un contenedor.

### E7. Revisión, comunicación y bitácora

1. Aplique la lista de revisión oficial (Tabla 3) a sus cinco vistas en `revision-c4.md` (sí/no y evidencia).
2. Use el **Prompt IA 2** como segundo revisor e identifique **al menos un falso positivo**.
3. Use el **Prompt IA 3** para redactar `explicacion.md` (120 palabras o menos) y corríjala: verifique que todo lo que dice sea cierto según su modelo.
4. Complete `bitacora-ia.md` con **5 o más interacciones** (fecha, herramienta, prompt, propuesta, verificación, decisión).

### E8. README, revisión cruzada y entrega

1. Agregue al `README.md` la sección **"Arquitectura C4 (Lab 06)"**, con el diagrama de contexto en Mermaid (copiado de `export/`) y enlaces a las imágenes de las cinco vistas.
2. **Revisión cruzada entre grupos:** cada grupo revisa el diagrama de contenedores de otro grupo con la lista oficial y deja sus observaciones en un *issue* de GitHub; el grupo revisado responde o corrige.
3. Cree la etiqueta `v0.6-c4` (`git tag v0.6-c4` y `git push origin v0.6-c4`) y entregue el enlace del repositorio en el Aula Virtual hasta el **sábado 17/10/2026 a las 23:59**.

**Reto opcional (+2 puntos, sin exceder la nota máxima): documentación viva**

```
Opción A: agregue al workspace.dsl las directivas !docs y !adrs para incluir su documentación y los
ADR del
Laboratorio 04, y muestre en Structurizr Lite cómo se navega de cada vista a sus decisiones.
Opción B: configure una GitHub Action que, en cada push, ejecute "structurizr validate" y regenere
las
imágenes de las vistas; el push debe fallar si el modelo tiene errores.
```

## Cronograma de la semana

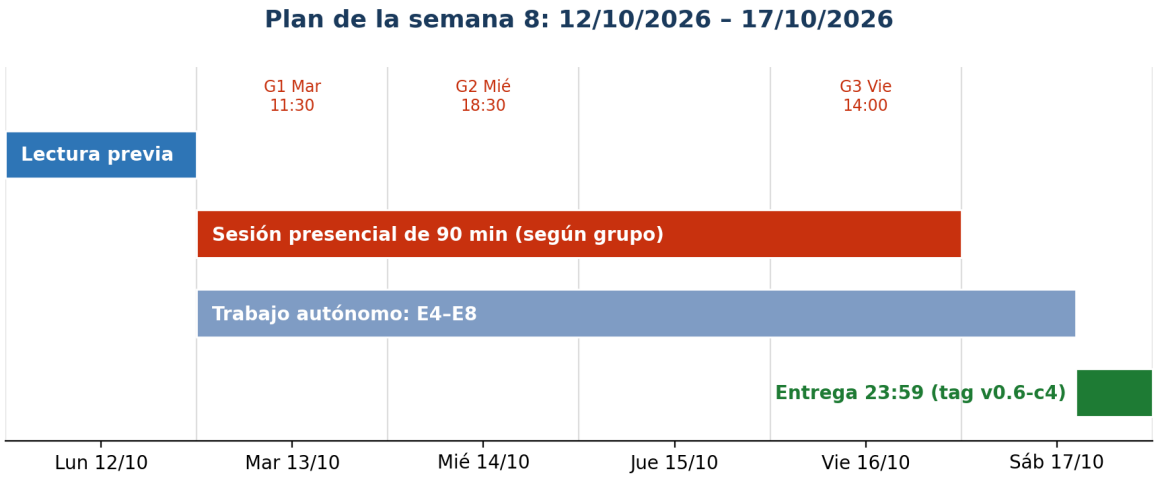



***Figura 10.*** *Plan de trabajo de la semana (12/10/2026 – 17/10/2026).*

| Fecha | Actividad |
|---|---|
| Lun 12/10/2026 | Publicación de la guía. Lectura previa: c4model.com (secciones Introduction, Abstractions y Diagrams) y el propio ADR-001. |
| Mar 13/10 – Vie 16/10 | Sesión de 90 min según el horario del grupo (Parte A: E1–E3) y checkpoint. |
| Día siguiente a la sesión | E4 (dinámica) y E5 (despliegue). |
| Antes del sábado | E6 (exportación y comparación) y E7 (revisión y bitácora). Revisión cruzada entre grupos. |
| Sáb 17/10/2026 | E8: README, etiqueta v0.6-c4 y entrega en el Aula Virtual hasta las 23:59. |

## IV. CUESTIONARIO

1. ¿Qué problema de los diagramas tradicionales de "cajas y flechas" resuelve el modelo C4? Dé un ejemplo de ambigüedad que haya evitado en su caso.
2. ¿Qué es un "contenedor" en C4 y por qué no equivale a un contenedor Docker? Indique un contenedor de su caso que no se ejecute en Docker.
3. ¿Qué público y qué nivel de detalle tiene cada uno de los cuatro niveles de C4? ¿Por qué rara vez se dibuja el nivel 4 a mano?
4. Compare la vista dinámica de C4 con el diagrama de secuencia UML de su flujo crítico. ¿Cuándo usaría cada una?
5. ¿Qué ventaja concreta tiene modelar con Structurizr DSL frente a dibujar cada vista con C4-PlantUML o Mermaid?
6. ¿Qué elementos del modelo C4 de su caso se justifican con el ADR-001 del Laboratorio 04? Cite al menos dos.
7. Describa un error de la IA al generar o revisar su modelo C4 y cómo lo detectó con la lista de revisión.
8. ¿Cómo adaptaría la explicación de su arquitectura para un docente evaluador, un inversionista y un nuevo developer del equipo?

## V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com

Brown, S. (s. f.). *The C4 model*. O'Reilly Media. https://www.oreilly.com/library/view/the-c4-model/9798341660113/

Brown, S. (s. f.). *Software architecture for developers* (Vol. 2: Visualise, document and explore your software architecture). Leanpub.

Structurizr. (s. f.). *Structurizr DSL: Language reference*. https://docs.structurizr.com/dsl

PlantUML Standard Library. (s. f.). *C4-PlantUML*. https://github.com/plantuml-stdlib/C4-PlantUML

Mermaid. (s. f.). *C4 diagrams*. https://mermaid.js.org/syntax/c4.html

ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description*.

Valente, M. T. (2020). *Engenharia de Software Moderna*, cap. 7: Arquitetura. https://engsoftmoderna.info/cap7.html

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

## TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

**TÉCNICAS:**

- Observación sistemática en el laboratorio (checkpoint E1–E3).
- Análisis de productos: modelo C4 como código, vistas exportadas y documentos.
- Revisión entre grupos mediante issues de GitHub.

**INSTRUMENTOS:**

- Lista de cotejo del checkpoint.
- Rúbrica analítica (criterios abajo).
- Lista de revisión oficial del modelo C4.
- Historial de commits y etiqueta v0.6-c4.

**CRITERIOS DE EVALUACIÓN**

| Criterio | Descripción | Puntaje |
|---|---|---|
| E1 Inventario y contexto | Inventario con origen; contexto con personas, sistemas externos y relaciones con intención. | 3 |
| E2 Contenedores | Contenedores coherentes con el ADR-001, con tecnología y protocolos. | 4 |
| E3 Componentes | Componentes coherentes con paquetes.puml del Lab 05 y sin ciclos. | 3 |
| E4 Vista dinámica | 4 o más pasos coherentes con la secuencia UML; comparación escrita. | 2 |
| E5 Despliegue | Nodos anidados con todas las instancias; coherente con el presupuesto. | 2 |
| E6 Exportación y comparación | Vistas exportadas e imágenes; contexto a mano y comparación fundamentada. | 2 |
| E7 Revisión y comunicación | Lista de revisión aplicada, falso positivo identificado, explicación clara y bitácora. | 2 |
| E8 Repositorio y entrega | README actualizado, issue de revisión cruzada atendido y etiqueta v0.6-c4 a tiempo. | 2 |
| Total | Reto opcional (documentación viva o GitHub Action): +2 puntos (nota máxima 20). | 20 |